

Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S11.

Mexi Banco: una verdad, varias copias.

Grupo: 2

Alumna:

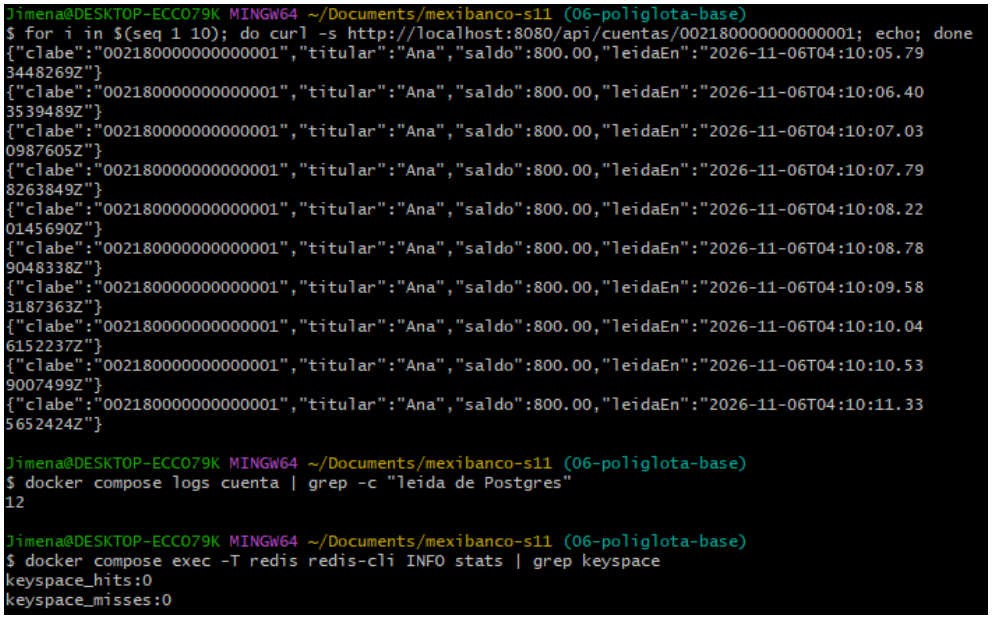
Valencia Lerdo Dalia Jimena

Cd. Universitaria a 7 de noviembre de 2026

1. **El punto de partida.**

Antes de modificar el código pedí diez veces el saldo de Ana. Cada respuesta trajo un leidaEn distinto, así que las diez se leyeron de Postgres. El log de las tres copias de cuenta registró 12 lecturas: las diez consultas más dos de la transferencia de prueba. Por otro lado, Redis estaba sin uso (keyspace_hits:0, keyspace_misses:0).

Imagen 1. Diez consultas a Postgres



2. **La caché.**

Qué hice.

ConfiguracionCache. @EnableCaching enciende las anotaciones de caché de Spring; sin esa línea, @Cacheable y @CacheEvict se ignoran sin avisar. Al caché cuentas le puse tres cosas: un TTL de 30 s (entryTtl), para que una copia vieja no viva para siempre; un serializador JSON de CuentaVista, para poder leer el valor con redis-cli en vez de ver bytes de Java; y transactionAware(), para que el borrado espere al commit.

ConsultaDeSaldo.consultar. @Cacheable con key = "#clabe": la clave queda cuentas::<clabe>. Si está en Redis (hit), contesta Redis y el método no corre. Si no (miss), lee Postgres y Spring guarda la copia con SET … PX 30000.

CuentaService.cargar y abonar. @CacheEvict con la misma clave borra la copia cuando cambia el saldo. Se borra en vez de actualizarse.

Lo que no toqué. El consultar de CuentaService, que usa cargar para decidir si alcanza el saldo, sigue leyendo siempre de Postgres. La consulta que muestra vive en ConsultaDeSaldo, otra clase, porque un @Cacheable llamado desde su propia clase no pasa por el proxy de Spring y se ignora.

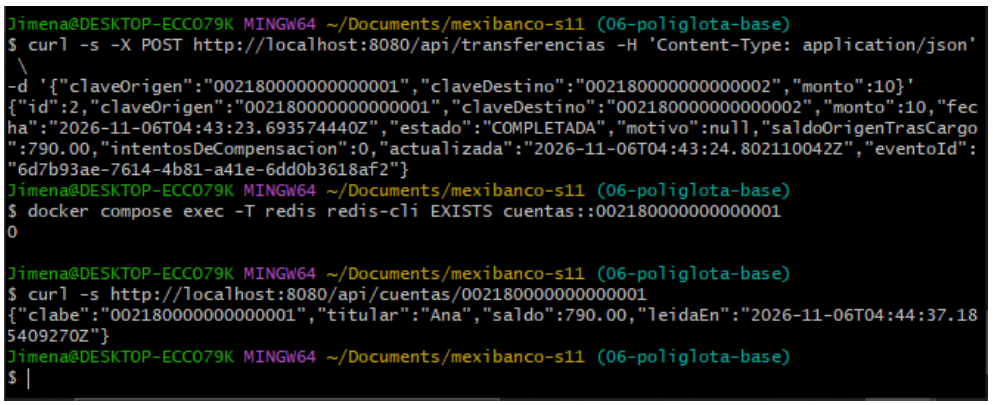
Imagen 2. Tres copias, una caché



En la Imagen 2 se ve que contestaron las tres copias de cuenta (los tres X-Instancia que se turnan), pero todas con el mismo leidaEn. Solo la primera consulta fue a Postgres y las otras cinco salieron de Redis, o sea que la caché no vive dentro de cada copia, es compartida. El valor guardado es un JSON que se lee directo y todavía le quedaban 26 segundos de vida.

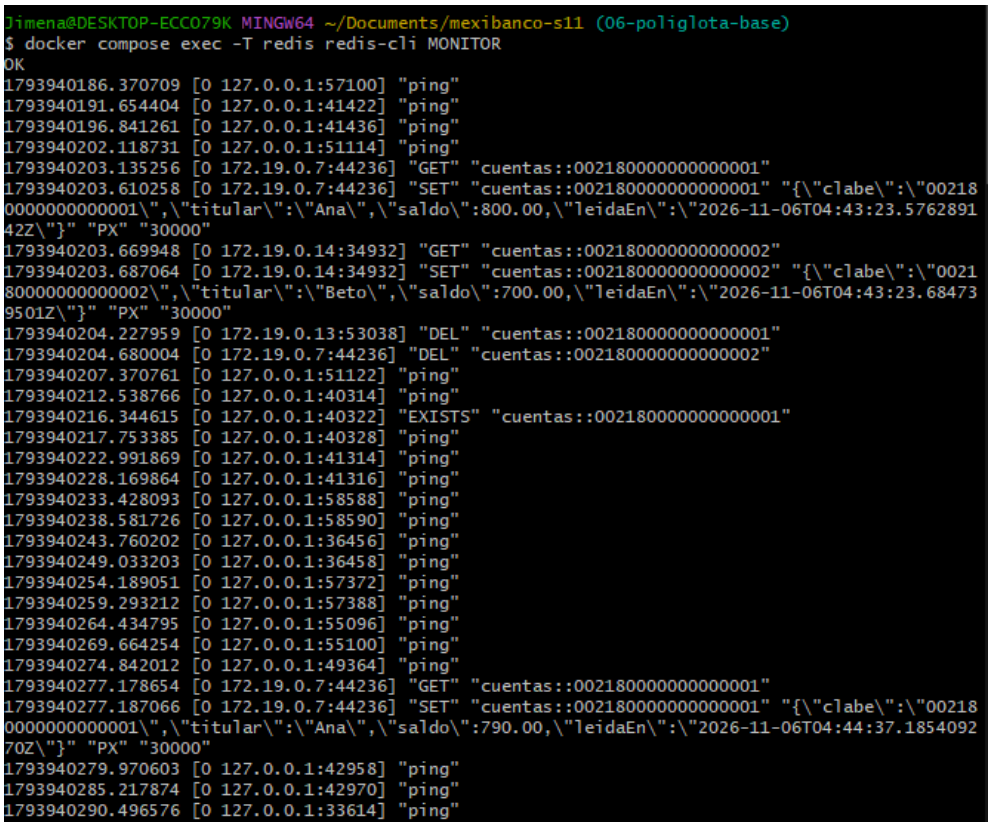
Después comprobé que la copia se borra. Transferí 10 de Ana a Beto y, como se ve en la Imagen 3, EXISTS dio 0 y la consulta siguiente ya trajo 790.00 con un leidaEn nuevo, es decir, volvió a leer Postgres.

Imagen 3. Transferencia y copia borrada



Con el MONITOR de Redis abierto vi el orden de todo. Primero la saga hizo un GET y un SET de Ana (con 800) y de Beto (con 700), porque sus copias estaban vacías. Luego aparecen el DEL de Ana y el de Beto, ya con el cargo y el abono confirmados. Al final está mi consulta: un GET que falló y un SET con 790 y PX 30000. Los ping son solo el chequeo de salud de Redis.

Imagen 4. MONITOR de Redis.



3. Mis medidas.

Hit ratio (hits / (hits + misses))

Escenario	Hits	Misses	Hit ratio
A: 100 consultas seguidas	98	2	98%
B: una transferencia de 1 peso cada 10 consultas	100	20	83.3%

En A esperaba 99 hits y 1 miss. El segundo miss fue por el TTL: las 100 consultas tardaron más de 30 s y la copia caducó una vez a mitad del ciclo. En B baja porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada uno falla.

Imagen 5. Hit ratio, escenario A

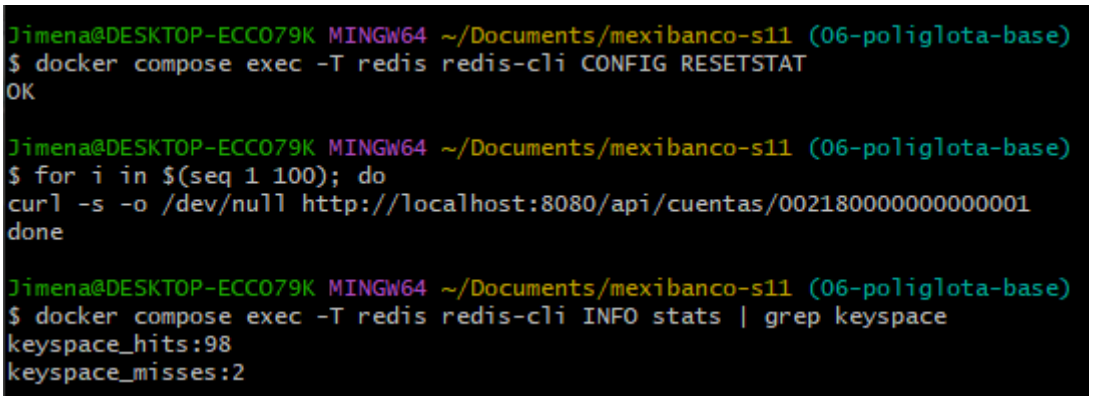
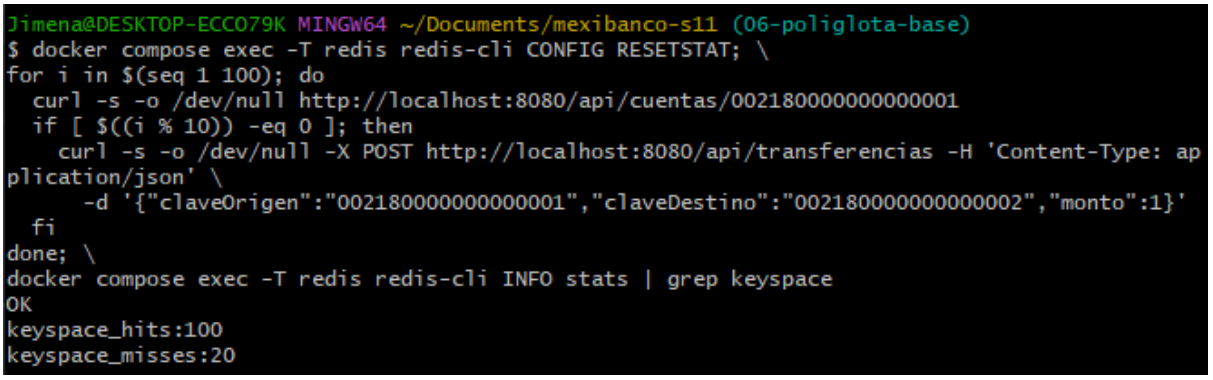


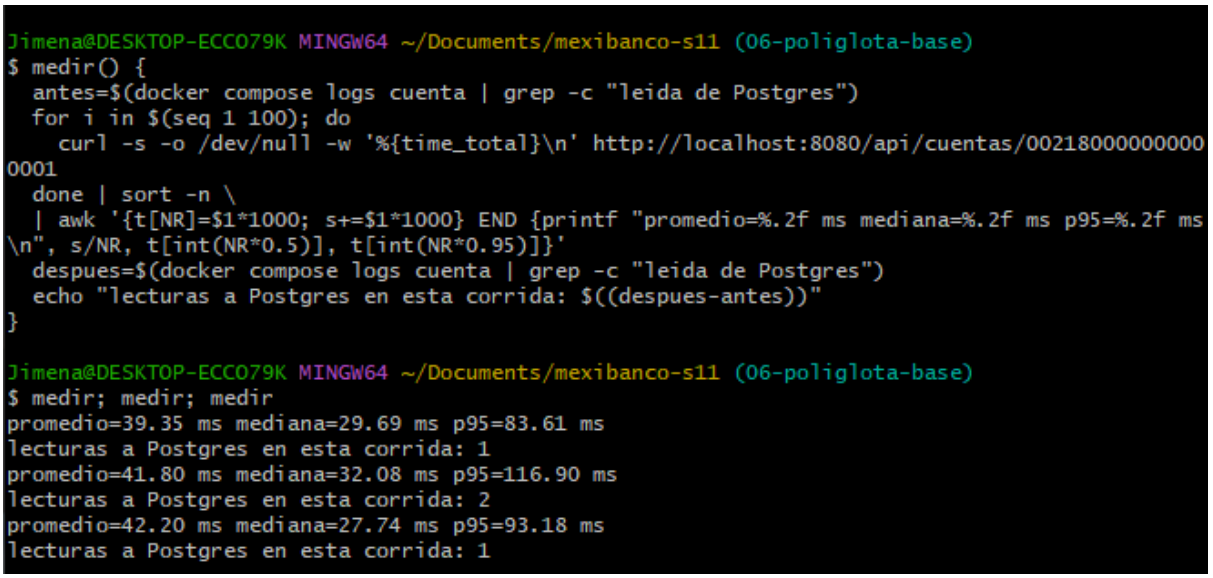
Imagen 6. Hit ratio, escenario B



Latencia.

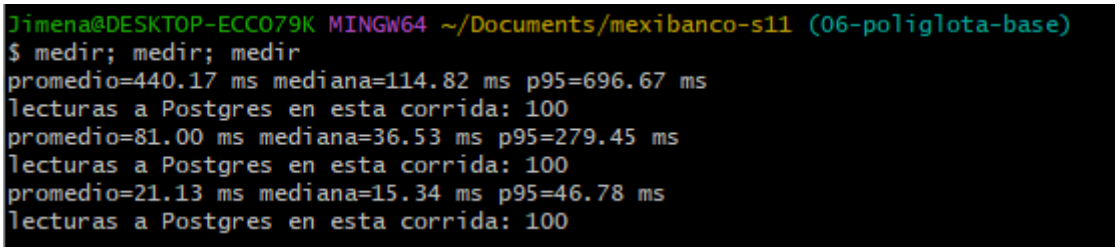
Para medir la latencia usé una función, medir, que hace 100 consultas del saldo de Ana, calcula promedio, mediana y p95, y cuenta cuántas veces se leyó Postgres. Hice tres corridas por tanda. Con la caché encendida, la primera tanda dio medianas de 29.69, 32.08 y 27.74 ms, promedios entre 39 y 42 ms y p95 de 83.61, 116.90 y 93.18 ms. Postgres se leyó solo 1, 2 y 1 veces por corrida: la lectura que llena la copia y, en la segunda, otra cuando caducó el TTL a medio ciclo.

Imagen 7. Latencia con caché, primera tanda



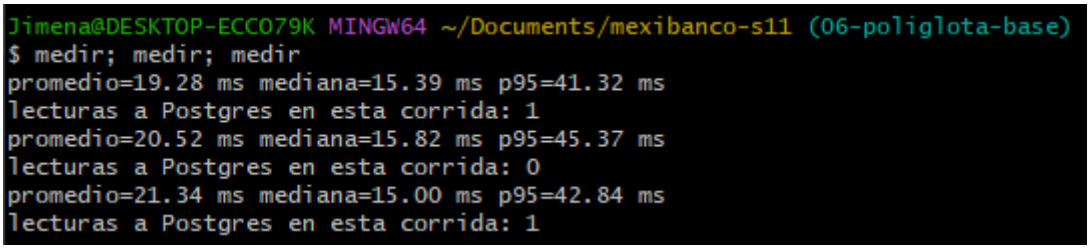
Luego apagué la caché con CACHE_TIPO=none y ahí cada consulta fue a Postgres: 100 lecturas en las tres corridas. La latencia, en cambio, empezó muy alta y fue bajando: mediana de 114.82 ms en la primera (promedio 440.17 y p95 696.67), 36.53 ms en la segunda (promedio 81.00 y p95 279.45) y 15.34 ms en la tercera (promedio 21.13 y p95 46.78). Creo que fue la JVM de cuenta, recién reiniciada, calentándose, y la laptop cargada. Por eso no comparé estas corridas con las de la primera tanda.

Imagen 8. Latencia sin caché



Volví a encender la caché, esperé a que las tres copias estuvieran healthy y calenté con unas consultas antes de repetir (Imagen 9). Salieron medianas de 15.39, 15.82 y 15.00 ms, promedios de 19.28, 20.52 y 21.34 ms y p95 de 41.32, 45.37 y 42.84 ms, con 1, 0 y 1 lecturas a Postgres.

Imagen 9. Latencia con caché, ya calentada



La comparación justa es esta última tanda contra la tercera corrida sin caché: la mediana es prácticamente la misma (unos 15 ms), pero Postgres pasó de 100 lecturas por corrida a casi ninguna. También hice una tanda sin caché que descarté, porque dio 0 lecturas a Postgres, algo que no tiene sentido sin caché, y en ese momento las copias de cuenta todavía estaban arrancando.

4. **Los avisos.**

Completé BitacoraDeAvisos: documento() arma cada aviso con su _id (eventoId:clabe), monto y saldo como Decimal128 y los campos de su canal; registrar() inserta uno por uno y, si Mongo dice que el _id ya existe, no lo cuenta; crearIndice() crea clabe_fecha; y ultimos() trae los 10 más recientes de una CLABE. En el consumidor cambié una sola línea para que guarde en Mongo y no en las dos bases.

Hice una transferencia de Ana a Beto (200) y otra de Ana a Carla (50). Quedaron cuatro avisos, cada uno con la forma de su canal (Imágenes 10 y 11): dos SMS de Ana con telefono y texto, un CORREO de Beto con correo, asunto y cuerpo, y un PUSH de Carla con token, titulo y cuerpo.

Imagen 10. Avisos en MongoDB

```
Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: 'd20f0107-2bb4-4b4c-aecd-44673d1edab9:002180000000000001',
    eventoId: 'd20f0107-2bb4-4b4c-aecd-44673d1edab9',
    transferenciaId: Long('13'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('580.00'),
    fecha: ISODate('2026-11-06T06:48:13.247Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 580.00'
  },
  {
    _id: 'd20f0107-2bb4-4b4c-aecd-44673d1edab9:002180000000000002',
    eventoId: 'd20f0107-2bb4-4b4c-aecd-44673d1edab9',
    transferenciaId: Long('13'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('920.00'),
    fecha: ISODate('2026-11-06T06:48:13.247Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 920.00'
  },
  {
    _id: 'c6a61cca-1e00-462b-90de-707d294118a9:002180000000000001',
    eventoId: 'c6a61cca-1e00-462b-90de-707d294118a9',
    transferenciaId: Long('14'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('530.00'),
    fecha: ISODate('2026-11-06T06:48:23.868Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 530.00'
  },
```

Imagen 11. Avisos por canal y filtro

```
  {
    _id: 'c6a61cca-1e00-462b-90de-707d294118a9:002180000000000001',
    eventoId: 'c6a61cca-1e00-462b-90de-707d294118a9',
    transferenciaId: Long('14'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('530.00'),
    fecha: ISODate('2026-11-06T06:48:23.868Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 530.00'
  },
  {
    _id: 'c6a61cca-1e00-462b-90de-707d294118a9:002180000000000003',
    eventoId: 'c6a61cca-1e00-462b-90de-707d294118a9',
    transferenciaId: Long('14'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('350.00'),
    fecha: ISODate('2026-11-06T06:48:23.868Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
  }
]

Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists:
true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: 'd20f0107-2bb4-4b4c-aecd-44673d1edab9:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'c6a61cca-1e00-462b-90de-707d294118a9:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Luego hice doce transferencias de 1 peso de Ana a Beto y la colección llegó a 28 avisos.

Imagen 12. Doce transferencias y 28 avisos

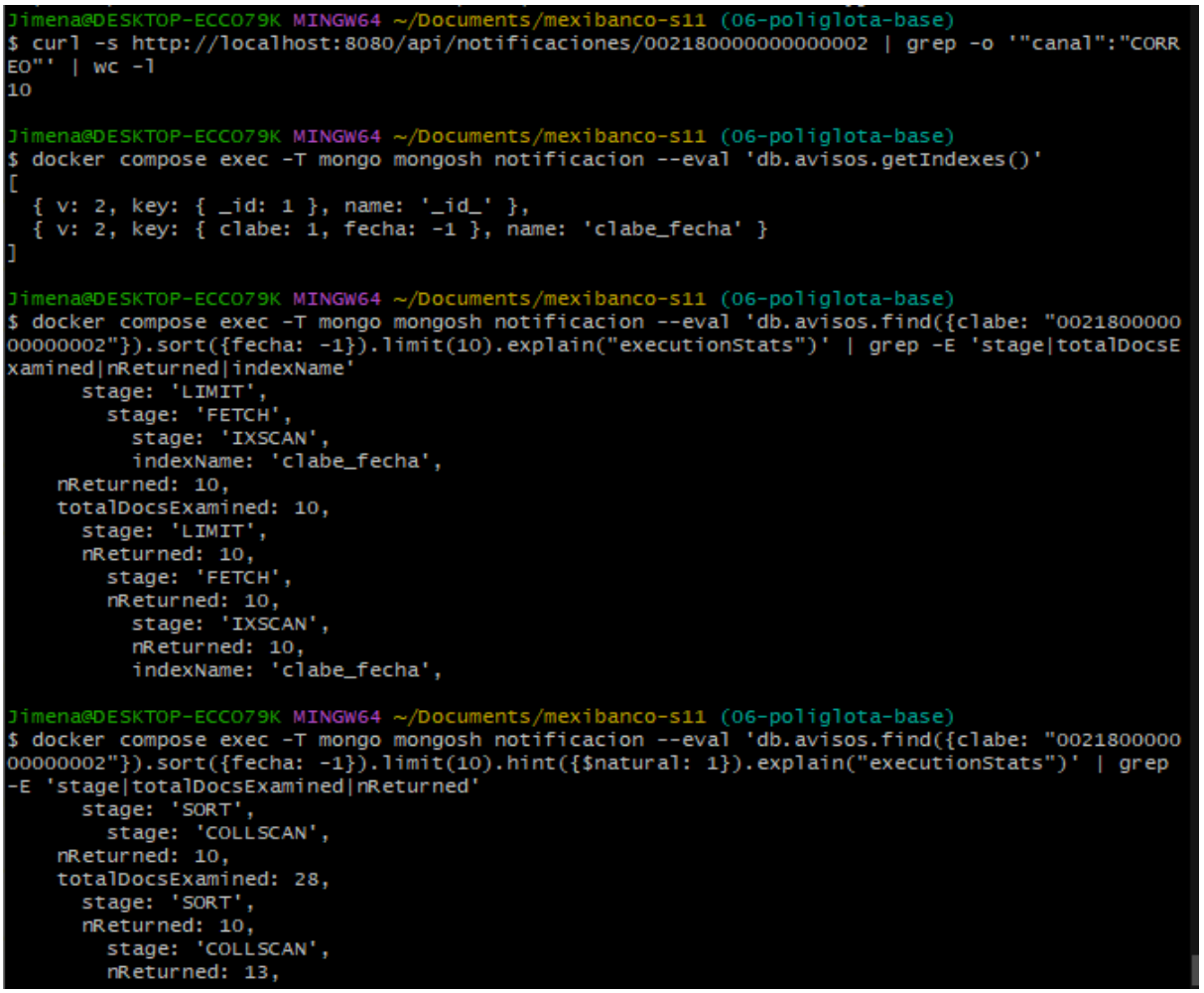
```
Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ for i in $(seq 1 12); do
  curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
    -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
done

Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ sleep 5

Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
28
```

Primero el endpoint de Beto: aunque tiene 13 avisos, devolvió exactamente 10, todos CORREO. Después los índices: además de id aparece clabe_fecha. Y por último los dos explain: con índice Mongo usó IXSCAN y examinó 10 documentos; obligándolo a no usarlo, hizo COLLSCAN con SORT y examinó los 28. Con 28 avisos casi no se nota, pero con 28 millones sería leer 10 contra 28 millones.

Imagen 13. Endpoint, índice y explain.



```
Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002 | grep -o '"canal":"CORR
EO"' | wc -l
10

Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]

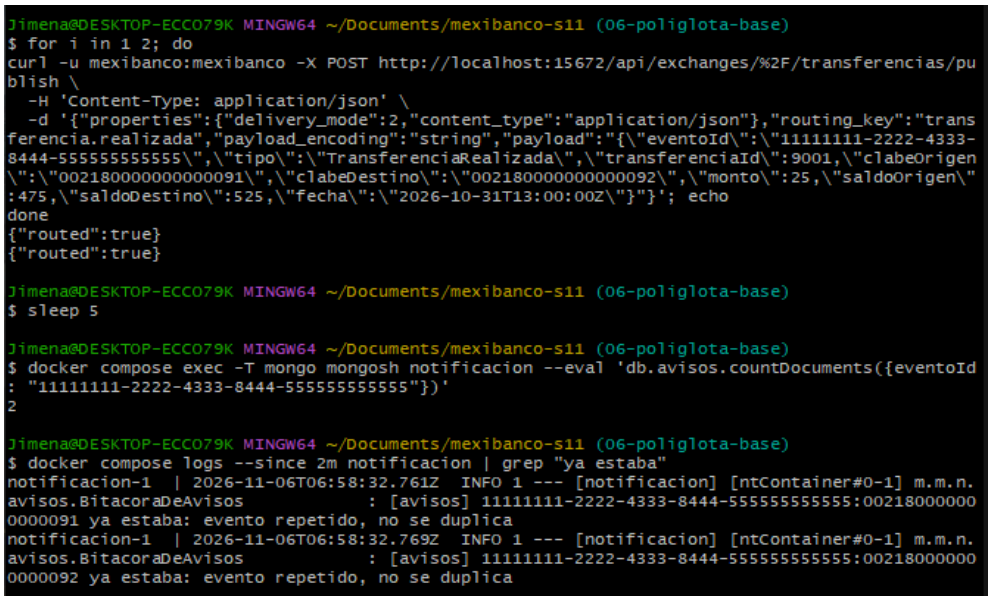
Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "0021800000
00000002"}).sort({fecha: -1}).limit(10).explain("executionStats")' | grep -E 'stage|totalDocsE
xamined|nReturned|indexName'
      stage: 'LIMIT',
        stage: 'FETCH',
          stage: 'IXSCAN',
          indexName: 'clabe_fecha',
    nReturned: 10,
    totalDocsExamined: 10,
      stage: 'LIMIT',
      nReturned: 10,
        stage: 'FETCH',
        nReturned: 10,
          stage: 'IXSCAN',
          nReturned: 10,
          indexName: 'clabe_fecha',

Jimena@DESKTOP-ECCO79K MINGW64 ~/Documents/mexibanco-s11 (06-poliglota-base)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "0021800000
00000002"}).sort({fecha: -1}).limit(10).hint({$natural: 1}).explain("executionStats")' | grep
-E 'stage|totalDocsExamined|nReturned'
      stage: 'SORT',
        stage: 'COLLSCAN',
    nReturned: 10,
    totalDocsExamined: 28,
      stage: 'SORT',
      nReturned: 10,
        stage: 'COLLSCAN',
        nReturned: 13,
```

5. **El duplicado y el dato viejo.**

El duplicado. La mensajería entrega al menos una vez, así que un evento puede llegar dos veces. Para provocarlo publiqué el mismo evento dos veces desde la API de RabbitMQ, con CLABEs inventadas (091 y 092). Mi predicción era que quedarían 2 documentos y no 4.

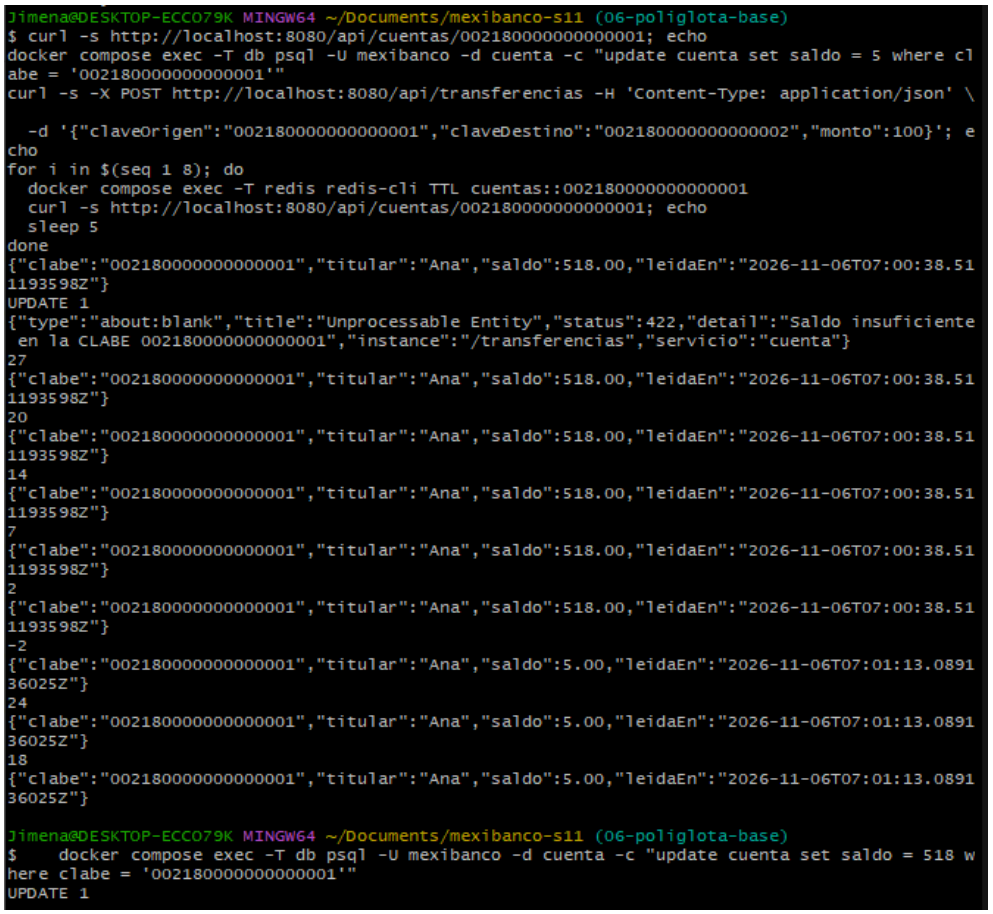
Imagen 14. Evento duplicado.



Los dos publish respondieron routed:true, el conteo dio 2 y el log mostró dos líneas “ya estaba: evento repetido, no se duplica”, una por CLABE. Pasa porque el _id es eventoId:clabe: la segunda vez Mongo rechaza el _id repetido con DuplicateKeyException y mi código la atrapa, no la cuenta y sigue. Por eso inserto de uno en uno: si fueran todos juntos, el primer repetido detendría el insert y, si el proceso se cayó a la mitad, la reentrega no guardaría el segundo aviso.

Ana tenía 518 y le cambié el saldo a 5 directo en Postgres, por fuera de la app, así que el @CacheEvict nunca corrió y la copia siguió en Redis. Predije que la app seguiría mostrando el saldo viejo y que la transferencia de 100 no pasaría, porque el cargo lee Postgres.

Imagen 15. Dato viejo en la caché



Mientras el TTL bajaba, de 27 a 2, el cliente siguió viendo 518.00 con el mismo leidaEn y la transferencia de 100 respondió 422 por saldo insuficiente. Cuando el TTL llegó a -2 la copia caducó y la siguiente consulta mostró 5.00 con un leidaEn nuevo. Entre el leidaEn viejo (07:00:38) y el nuevo (07:01:13) pasaron unos 34 s: los 30 s del TTL más lo que tardó mi ciclo en volver a preguntar, que era cada 5 s. Al final regresé el saldo de Ana a 518 con otro UPDATE; este cambio rompe a propósito el cuadre entre el saldo y los movimientos, así que no se debe dejar así.

6. Preguntas de cierre.

¿ Por qué MongoDB para los avisos y no para el saldo? Usa lo que viste: la forma de cada aviso, cómo se leen y qué tendría que garantizar el saldo.

Cada aviso tiene forma distinta según su canal (teléfono y texto, correo con asunto, token de push), se escribe una vez, nunca se actualiza y se lee entero por CLABE. No hay joins ni reglas entre avisos, así que un documento por aviso encaja bien y no hace falta una columna por campo de cada canal. El saldo es dinero: tiene que cambiar con transacciones y con versión (@Version), sin perderse ni contradecirse, y esas garantías las da Postgres.

¿Qué hit ratio mediste en A y en B, y por qué cambia? Con tus números de latencia y de lecturas a Postgres: ¿qué compraste con la caché, velocidad o capacidad?

Medí 98 % en A (98 hits, 2 misses) y 83.3 % en B (100 hits, 20 misses). Baja porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada una es un miss: más escrituras significan más misses. Con la JVM ya caliente, la latencia casi no cambió (mediana 15.34 ms sin caché y 15.00 a 15.82 ms con caché), pero las lecturas a Postgres bajaron de 100 a entre 0 y 2 por cada 100 consultas. Compré capacidad, no velocidad: Postgres, que tiene que aguantar los cargos y abonos, recibe unas 100 veces menos lecturas.

¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? Usa lo que viste en la Parte 4.

En la Parte 4 cambié el saldo de Ana a 5 en Postgres y la app siguió mostrando 518.00 unos 34 s. Como el cargo lee Postgres, la transferencia de 100 falló con 422. Si hubiera leído la caché, habría visto 518, habría aprobado el cargo y Ana habría quedado en -95: dinero que no existía. Por eso la caché muestra, pero nunca decide.

Al aplicar un movimiento, ¿por qué se hace DEL de la copia y no SET con el saldo nuevo? ¿Y por qué después del commit y no antes?

Con DEL, la siguiente consulta reconstruye la copia desde Postgres, que es la verdad. Con SET tendría que calcular el saldo nuevo y guardarlo, y si la transacción se revierte o dos escrituras se cruzan, la caché se quedaría con un valor que Postgres nunca tuvo. Va después del commit porque, si se borra antes, otra consulta puede leer todavía el saldo viejo de Postgres y volver a guardarlo en la caché. En el MONITOR la copia de Ana con 800, que dejó la comprobación de la saga, se borró hasta después del commit, y la siguiente consulta ya trajo 790. Eso es lo que hace transactionAware().